

Методические материалы по лабораторным работам курса «Fullstack»

Первый семестр

Общие требования

В течение семестра разрабатывается один проект по теме, согласованной с преподавателем. Каждая лабораторная работа продолжает предыдущую. Итогом является работоспособное full-stack приложение (MVP).

Обязательный стек

1. **Backend:** Python + FastAPI.
2. **База данных и ORM:** PostgreSQL + SQLAlchemy.
3. **Frontend:** React + TypeScript.
4. **Контроль версий:** Git. Платформу размещения репозитория определяет преподаватель.

Порядок выполнения и сдачи

1. Лабораторные работы выполняются и сдаются строго последовательно, с №1 по №5. Результат каждого этапа должен быть зафиксирован в репозитории.
2. Приложение должно включать несколько функционально различных экранов и пользовательских сценариев.
3. **Если результата работы нет в репозитории, работа не проверяется.** Репозиторий должен быть доступен преподавателю к моменту сдачи.
4. Ведите историю разработки: регулярно фиксируйте завершённые изменения и используйте осмысленные сообщения коммитов. Единственная загрузка готового проекта перед сдачей не заменяет историю разработки.
5. Поддерживайте README с описанием проекта, зависимостей, настройки окружения и порядка запуска. Добавьте .gitignore; пароли, ключи и другие секреты не размещайте в репозитории.
6. **Генерация программного кода с помощью искусственного интеллекта запрещена.** Запрет распространяется на создание проекта целиком и отдельных фрагментов кода, включая ИИ-автодополнение. При сдаче продемонстрируйте результат лабораторной работы и объясните реализованные решения.

Лабораторная работа №1

Интерфейс приложения и каркас frontend

Цели лабораторной работы

1. Определить основные экраны и пользовательские сценарии проекта.
2. Создать первоначальную версию интерфейса на React + TypeScript с использованием библиотеки UI-компонентов.

Задание

1. Определите назначение приложения, основные пользовательские сценарии и необходимые экраны. Кратко опишите их в README.
2. Создайте frontend-приложение на React + TypeScript и организуйте начальную структуру компонентов и страниц.
3. Реализуйте верстку основных экранов и переходы между ними с помощью клиентской маршрутизации. На этом этапе допускается использование демонстрационных данных.
4. Выберите и используйте готовую библиотеку UI-компонентов. Обеспечьте единое оформление элементов, читаемость текста и базовую адаптивность интерфейса.
5. Сохраните скриншоты всех реализованных экранов в репозитории, например в каталоге docs/screenshots. При последующих изменениях интерфейса обновляйте скриншоты и кратко фиксируйте причины существенного изменения или упрощения в README.
6. Проверьте запуск приложения и навигацию между экранами. Зафиксируйте результат в Git и добавьте инструкцию запуска frontend в README.

Лабораторная работа №2

Backend, API и работа с базой данных

Цели лабораторной работы

1. Организовать серверную часть проекта на FastAPI.
2. Реализовать модель данных и CRUD-операции с использованием PostgreSQL и SQLAlchemy.

Задание

1. Создайте backend-приложение и разделите код по назначению:
 - 1.1. маршруты API; модели SQLAlchemy и схемы валидации;
 - 1.2. подключение и сессии БД; операции с данными и бизнес-логика;
 - 1.3. конфигурация приложения.
2. Подключите PostgreSQL через SQLAlchemy. Вынесите параметры подключения в конфигурацию окружения и подготовьте пример настроек без секретов.
3. Определите основные сущности проекта, их поля и связи. Реализуйте связанные таблицы с первичными и внешними ключами. Кратко опишите модель данных в README.
4. Реализуйте создание, чтение, изменение и удаление данных для основных сущностей проекта. Операции должны работать с PostgreSQL через SQLAlchemy.
5. Добавьте схемы валидации запросов и ответов. Обеспечьте понятные сообщения об ошибках и подходящие HTTP-статусы, в том числе для некорректных данных и отсутствующих записей.
6. Проверьте операции API и сохранение данных в БД. Убедитесь, что связанные записи обрабатываются корректно.
7. Зафиксируйте результат в Git. Дополните README порядком настройки и запуска backend, подготовки БД и создания таблиц.

Лабораторная работа №3

Архитектура frontend и динамический интерфейс

Цели лабораторной работы

1. Реализовать интерактивное поведение основных экранов приложения.
2. Организовать понятную модульную структуру frontend.

Задание

1. Разделите код приложения на страницы, пользовательские действия, данные предметной области и общие компоненты. Обеспечьте понятное назначение модулей и отсутствие дублирования.
2. В качестве ориентира рекомендуется упрощённая структура на основе Feature-Sliced Design (FSD): app - настройка приложения; pages - страницы; features - пользовательские действия; entities - сущности; shared - общие компоненты и утилиты. Полное соблюдение FSD не требуется. Кратко опишите выбранную структуру в README.
3. Реализуйте основные интерактивные элементы: формы, управление состоянием компонентов и реакцию интерфейса на действия пользователя. Добавьте типизацию данных и свойств компонентов средствами TypeScript.
4. Добавьте клиентскую валидацию форм и понятное отображение ошибок ввода. Для экранов с данными предусмотрите состояния загрузки, ошибки и отсутствия данных.
5. Добавьте осмысленную анимацию или переходы интерфейса, например при открытии формы, переключении состояния или появлении элемента.
6. Проверьте пользовательские сценарии и сохранение ранее реализованной навигации. Для проверки динамического поведения допускаются демонстрационные данные до полной интеграции с API.
7. Зафиксируйте результат в Git, обновите README и скриншоты изменённых экранов.

Лабораторная работа №4

Аутентификация и авторизация

Цели лабораторной работы

1. Реализовать совместную работу frontend и backend при входе пользователя.
2. Организовать доступ к защищённым ресурсам с использованием access и refresh токенов.

Задание

1. Реализуйте регистрацию и вход пользователя. Пароли должны храниться в БД в виде стойких хешей, предназначенных для хранения паролей.
2. Реализуйте выдачу двух токенов: access token для доступа к защищённому API и refresh token для обновления access token. Используйте JWT для access token и задайте сроки действия обоих токенов.
3. Реализуйте обновление access token по действующему refresh token. Предусмотрите отказ при некорректном, истёкшем или отозванном refresh token.
4. Добавьте получение текущего пользователя и защиту необходимых API-маршрутов. Разграничьте доступ к публичным и приватным ресурсам; для пользовательских данных проверяйте право на выполняемую операцию.
5. Реализуйте выход из приложения: очистку состояния пользователя на клиенте и прекращение возможности обновления access token с помощью refresh token текущей сессии.

Лабораторная работа №5

Интеграция frontend и backend

Цели лабораторной работы

1. Объединить ранее реализованные части проекта в работоспособное приложение.
2. Проверить основные пользовательские сценарии с реальными данными.

Задание

1. Подключите основные экраны frontend к API на FastAPI. В завершённых пользовательских сценариях замените демонстрационные данные данными из PostgreSQL.
2. Реализуйте через интерфейс просмотр, создание, редактирование и удаление данных в соответствии с функциональностью проекта и правами пользователя.
3. Согласуйте форматы запросов и ответов между frontend и backend. После изменения данных обеспечьте актуальное отображение результата в интерфейсе.
4. Подключите состояния загрузки, ошибки и отсутствия данных к реальным запросам. Отображайте ошибки валидации и отказы в доступе понятным для пользователя образом.
5. Проверьте приложение целиком: навигацию, работу форм, операции с данными, вход и выход. Исправьте ошибки, мешающие выполнению основных сценариев.
6. Зафиксируйте итоговое состояние проекта в Git, обновите скриншоты и README. Проверьте запуск frontend, backend и БД по инструкции из репозитория и продемонстрируйте работоспособный MVP.